CASE 1

WRITE

OPERATION

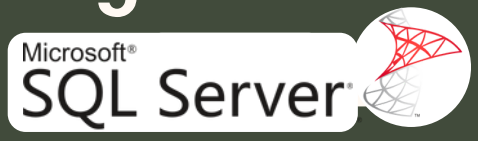
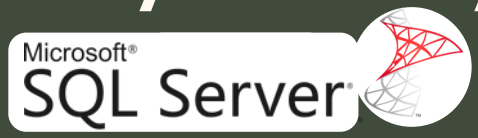
Microsoft® SQL Server®

Integration

With

AND

jupyter

This document outlines a project demonstrating the integration between  and  using  development environment. The core objective is to showcase how  can be leveraged to dynamically interact with a  database, focusing specifically on data insertion and writing operations.

1. Context and Overview

The focus is on the data writing and insertion part, using variables to make the registration process dynamic and automated.
Through the pyodbc library, we can establish a direct connection between and allowing the execution of SQL commands (INSERT, UPDATE, DELETE, etc.) in a simple and efficient way.

2. Preparing Microsoft SQL Server for Integration

This step focused on preparing the target database in Microsoft SQL Server. A new database, PythonSQL, was created to house the project's data. We then defined the Sales table with the necessary columns (sale_id, sale_date, customer, etc.). The process concluded with the successful insertion of a manual test record to validate the table's structure and ensure it was ready for the automated data insertion process via Python.

View Code

SQLQuery1.sql - L...M4NC\samsung (68))*

```sql
CREATE DATABASE PythonSQL;
USE PythonSQL;

-- Creating the table and columns that will receive the data
CREATE TABLE Sales(
    sale_id INT,
    sale_date DATE,
    customer VARCHAR(100),
    product VARCHAR(100),
    price DECIMAL(10, 2),
    quantity INT
);
```

2. Preparing Microsoft SQL Server for Integration

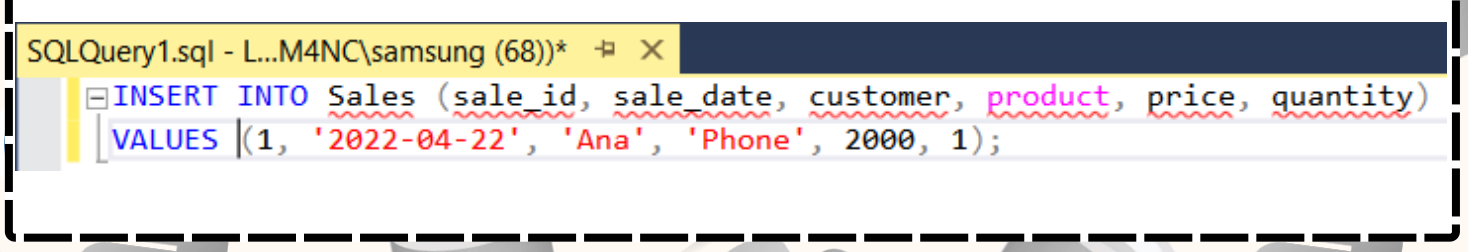
View Code

```
CREATE DATABASE PythonSQL;
USE PythonSQL;

-- Creating the table and columns that will receive the data
CREATE TABLE Sales(
    sale_id INT,
    sale_date DATE,
    customer VARCHAR(100),
    product VARCHAR(100),
    price DECIMAL(10, 2),
    quantity INT
);
```

✅ **Database and table successfully created.**
Now, let's add a first test record:

```
INSERT INTO Sales (sale_id, sale_date, customer, product, price, quantity)
VALUES (1, '2022-04-22', 'Ana', 'Phone', 2000, 1);
```



3. Initial Configurations in

The integration script will later be saved in a specific folder.

In Jupyter Notebook, create a new Python 3 file and start writing the connection code.

View Code

Installing the integration library:

```
!pip install pyodbc
```

```
In [1]: pip install pyodbc
```

Setting up the database connection:

```
import pyodbc

connection_data = (
   "Driver={SQL Server};"
   "Server=LAPTOP-SRP0M4NC;"
   "Database=PythonSQL;"
)

connection = pyodbc.connect(connection_data)
print("Connection successful!")
```

```
In [2]: import pyodbc

        connection_data = (
            "Driver={SQL Server};"
            "Server=LAPTOP-SRP0M4NC;"
            "Database=PythonSQL;"
        )

        connection = pyodbc.connect(connection_data)
        print("Connection successful!")
```

- The Driver parameter defines the SQL Server connector.
  - Server indicates the machine or host name.
  - Database specifies which database will be used.

This configuration allows direct integration between Python and SQL Server

4. Adding SQL Commands inside jupyter

Now let's insert SQL commands directly from Python to write data into the table:

View Code

```
cursor = connection.cursor()

command = """INSERT INTO Sales (sale_id, sale_date, customer, product, price, quantity)
VALUES (1, '2022-04-22', 'Ana', 'Phone', 2000, 1)"""

cursor.execute(command)
cursor.commit()
```

```
In [2]: cursor = connection.cursor()

        command = """INSERT INTO Sales (sale_id, sale_date, customer, product, price, quantity)
        VALUES (1, '2022-04-22', 'Ana', 'Phone', 2000, 1)"""

        cursor.execute(command)
        cursor.commit()
```

Four more insertion examples:

```
In [2]: cursor = connection.cursor()

        command = """INSERT INTO Sales (sale_id, sale_date, customer, product, price, quantity)
        VALUES (3, '2022-09-10', 'Mariana', 'Headphones', 350, 2)"""

        cursor.execute(command)
        cursor.commit()
```

This configuration allows direct integration between Python and SQL Server

4. Adding SQL Commands inside

Four more insertion examples:

View Code

```
command = """INSERT INTO Sales (sale_id, sale_date, customer, product, price, quantity)
VALUES (2, '2022-07-01', 'Pedro', 'Laptop', 6000, 1)"""
cursor.execute(command)
cursor.commit()

command = """INSERT INTO Sales (sale_id, sale_date, customer, product, price, quantity)
VALUES (3, '2022-09-10', 'Mariana', 'Headphones', 350, 2)"""
cursor.execute(command)
cursor.commit()

command = """INSERT INTO Sales (sale_id, sale_date, customer, product, price, quantity)
VALUES (4, '2022-11-25', 'Lucas', 'Smart TV', 3200, 1)"""
cursor.execute(command)
cursor.commit()

command = """INSERT INTO Sales (sale_id, sale_date, customer, product, price, quantity)
VALUES (5, '2023-01-12', 'Julia', 'Digital Camera', 1800, 1)"""
cursor.execute(command)
cursor.commit()
```

5. Making Data Registration Automatic with Variables

To make the process dynamic, we can use Python variables to automatically register new data:

View Code

```
cursor = connection.cursor()

sale_id = 6
sale_date = "2023-06-17"
customer = "Diego"
product = "Tablet"
price = 1200
quantity = 1

command = f"""INSERT INTO Sales (sale_id, sale_date, customer, product, price, quantity)
VALUES ({sale_id}, '{sale_date}', '{customer}', '{product}', {price}, {quantity})"""

cursor.execute(command)
cursor.commit()

print("Data successfully inserted!")
```

```
In [3]: cursor = connection.cursor()

        sale_id = 6
        sale_date = "2023-06-17"
        customer = "Diego"
        product = "Tablet"
        price = 1200
        quantity = 1

        command = f"""INSERT INTO Sales (sale_id, sale_date, customer, product, price, quantity)
        VALUES ({sale_id}, '{sale_date}', '{customer}', '{product}', {price}, {quantity})"""

        cursor.execute(command)
        cursor.commit()

        print("Data successfully inserted!")
```

5. Conclusion

The  and  Integration Project (Write Operation) demonstrates in practice how to:

- Connect Python directly to SQL Server using pyodbc;
- Execute SQL commands inside Jupyter Notebook;
- And automate data insertion using Python variables.

This integration makes the process of writing and inserting data faster, more flexible, and programmable, making it ideal for projects involving data analysis, automated registration, or corporate database updates.

Next Steps

This final phase will establish the connection between  and the ContosoRetailDW database to read and extract data. The goal is to perform essential data analysis and calculate Key Performance Indicators (KPIs), concluding with the creation of analytical charts and visualizations directly within the  environment.